

```
!pip install keras-tuner --upgrade
```

```
Requirement already satisfied: keras-tuner in c:\users\kamal\anaconda3\envs\myenv\lib\site-packages (1.4.6)
Requirement already satisfied: keras in c:\users\kamal\anaconda3\envs\myenv\lib\site-packages (from keras-tuner) (2.13.1)
Requirement already satisfied: packaging in c:\users\kamal\appdata\roaming\python\python38\site-packages (from keras-tuner) (23.2)
Requirement already satisfied: requests in c:\users\kamal\anaconda3\envs\myenv\lib\site-packages (from keras-tuner) (2.31.0)
Requirement already satisfied: kt-legacy in c:\users\kamal\anaconda3\envs\myenv\lib\site-packages (from keras-tuner) (1.0.5)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\kamal\appdata\roaming\python\python38\site-packages (from requests->keras-tuner) (3.3.0)
Requirement already satisfied: idna<4,>=2.5 in c:\users\kamal\anaconda3\envs\myenv\lib\site-packages (from requests->keras-tuner) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\kamal\anaconda3\envs\myenv\lib\site-packages (from requests->keras-tuner) (1.26.16)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\kamal\anaconda3\envs\myenv\lib\site-packages (from requests->keras-tuner) (2023.7.22)
```

```
%matplotlib inline
```

```
import os
seed = 42
os.environ['PYTHONHASHSEED']=str(seed)
```

```
import pandas as pd
import numpy as np
import random
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GroupShuffleSplit
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout, Masking, TimeDistributed
from keras.models import Sequential
from keras.layers import Dense, Conv1D
from keras.layers import Dropout, Input
from keras import initializers
from keras.layers import MaxPooling1D
```

```

from keras.models import Model
from keras.layers import TimeDistributed, Flatten
from sklearn.model_selection import GroupShuffleSplit
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv1D, Dropout,
MaxPooling1D, TimeDistributed, Flatten, LSTM, Dense, Masking,
Concatenate
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint,
ReduceLROnPlateau
from tensorflow.keras import regularizers
from tensorflow.keras.optimizers import Adam, RMSprop
from kerastuner.tuners import Hyperband
from kerastuner.engine.hyperparameters import HyperParameters

```

```

dir_path = 'C:\\Users\\Kamal\\Documents\\AUC\\thesiswork\\env_eweda\\
CMaps'
os.chdir(dir_path)
print(os.getcwd())
train_file = 'train_FD003.txt'
print((dir_path+train_file))

```

```

C:\Users\Kamal\Documents\AUC\thesiswork\env_eweda\CMaps
C:\Users\Kamal\Documents\AUC\thesiswork\env_eweda\CMapstrain_FD003.txt

```

```

random.seed(seed)
np.random.seed(seed)
tf.random.set_seed(seed)
dir_path = 'C:\\Users\\Kamal\\Documents\\AUC\\thesiswork\\env_eweda\\
CMaps'
train_file = '\\train_FD003.txt'
test_file = '\\test_FD003.txt'
index_names = ['unit_nr', 'time_cycles']
setting_names = ['setting_1', 'setting_2', 'setting_3']
sensor_names = ['s_{}'.format(i+1) for i in range(0,21)]
col_names = index_names + setting_names + sensor_names
train = pd.read_csv((dir_path+train_file), sep='\\s+', header=None,
                    names=col_names)
test = pd.read_csv((dir_path+test_file), sep='\\s+', header=None,
                  names=col_names)
y_test = pd.read_csv((dir_path+'\\RUL_FD003.txt'), sep='\\s+',
                    header=None,
                    names=['RemainingUsefulLife'])
print(train.shape)
train.head()

```

```
(24720, 26)
```

	unit_nr	time_cycles	setting_1	setting_2	setting_3	s_1
s_2 \						
0	1	1	-0.0005	0.0004	100.0	518.67

642.36							
1	1	2	0.0008	-0.0003	100.0	518.67	
642.50							
2	1	3	-0.0014	-0.0002	100.0	518.67	
642.18							
3	1	4	-0.0020	0.0001	100.0	518.67	
642.92							
4	1	5	0.0016	0.0000	100.0	518.67	
641.68							

	s_3	s_4	s_5	...	s_12	s_13	s_14	s_15
s_16	s_17	\						
0	1583.23	1396.84	14.62	...	522.31	2388.01	8145.32	8.4246
0.03	391							
1	1584.69	1396.89	14.62	...	522.42	2388.03	8152.85	8.4403
0.03	392							
2	1582.35	1405.61	14.62	...	522.03	2388.00	8150.17	8.3901
0.03	391							
3	1585.61	1392.27	14.62	...	522.49	2388.08	8146.56	8.3878
0.03	392							
4	1588.63	1397.65	14.62	...	522.58	2388.03	8147.80	8.3869
0.03	392							

	s_18	s_19	s_20	s_21
0	2388	100.0	39.11	23.3537
1	2388	100.0	38.99	23.4491
2	2388	100.0	38.85	23.3669
3	2388	100.0	38.96	23.2951
4	2388	100.0	39.14	23.4583

[5 rows x 26 columns]

```
def add_remaining_useful_life(df):
    # Get the total number of cycles for each unit
    grouped_by_unit = df.groupby(by="unit_nr")
    max_cycle = grouped_by_unit["time_cycles"].max()

    # Merge the max cycle back into the original frame
    result_frame = df.merge(max_cycle.to_frame(name='max_cycle'),
left_on='unit_nr', right_index=True)

    # Calculate remaining useful life for each row
    remaining_useful_life = result_frame["max_cycle"] -
result_frame["time_cycles"]
    result_frame["RUL"] = remaining_useful_life

    # drop max_cycle as it's no longer needed
    result_frame = result_frame.drop("max_cycle", axis=1)
    return result_frame
```

```
train = add_remaining_useful_life(train)
train[index_names+['RUL']].head()
```

	unit_nr	time_cycles	RUL
0	1	1	258
1	1	2	257
2	1	3	256
3	1	4	255
4	1	5	254

```
X_train = train[setting_names + sensor_names].copy()
y_train = train['RUL'].copy()
y_train_clipped = y_train.clip(upper=125)
```

```
X_test = test.drop('time_cycles',
axis=1).groupby('unit_nr').last().copy() # get last row of each engine
```

```
def evaluate(y_true, y_hat, label='test'):
    mse = mean_squared_error(y_true, y_hat)
    rmse = np.sqrt(mse)
    variance = r2_score(y_true, y_hat)
    print('{} set RMSE:{}, R2:{}'.format(label, rmse, variance))
lm = LinearRegression()
lm.fit(X_train, y_train_clipped)
```

```
# predict and evaluate
```

```
y_hat_train = lm.predict(X_train)
evaluate(y_train_clipped, y_hat_train, 'train')
```

```
y_hat_test = lm.predict(X_test)
evaluate(y_test, y_hat_test)
```

```
train set RMSE:19.329629974927535, R2:0.7736229075130826
test set RMSE:22.288215837615585, R2:0.7101043061544098
```

```
# before we start plotting, lets implement the condition specific scaling
```

```
def add_operating_condition(df):
    df_op_cond = df.copy()
```

```
    df_op_cond['setting_1'] = df_op_cond['setting_1'].round()
    df_op_cond['setting_2'] =
df_op_cond['setting_2'].round(decimals=2)
```

```
# converting settings to string and concatenating makes the operating condition into a categorical variable
```

```
    df_op_cond['op_cond'] = df_op_cond['setting_1'].astype(str) + '_'
+ \
                                df_op_cond['setting_2'].astype(str) + '_' + \
                                df_op_cond['setting_3'].astype(str)
```

```
    return df_op_cond
```

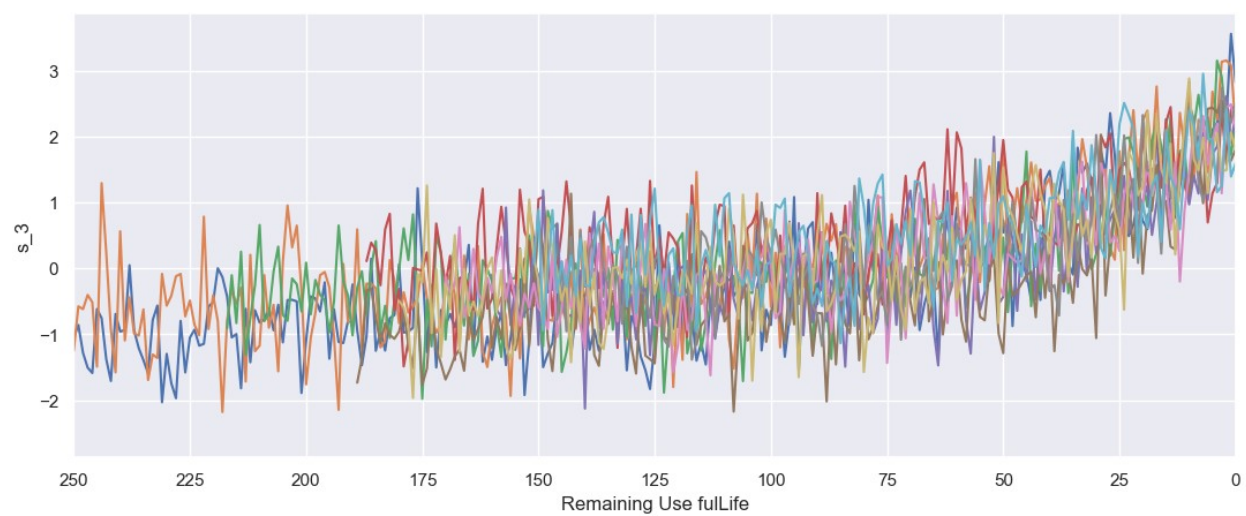
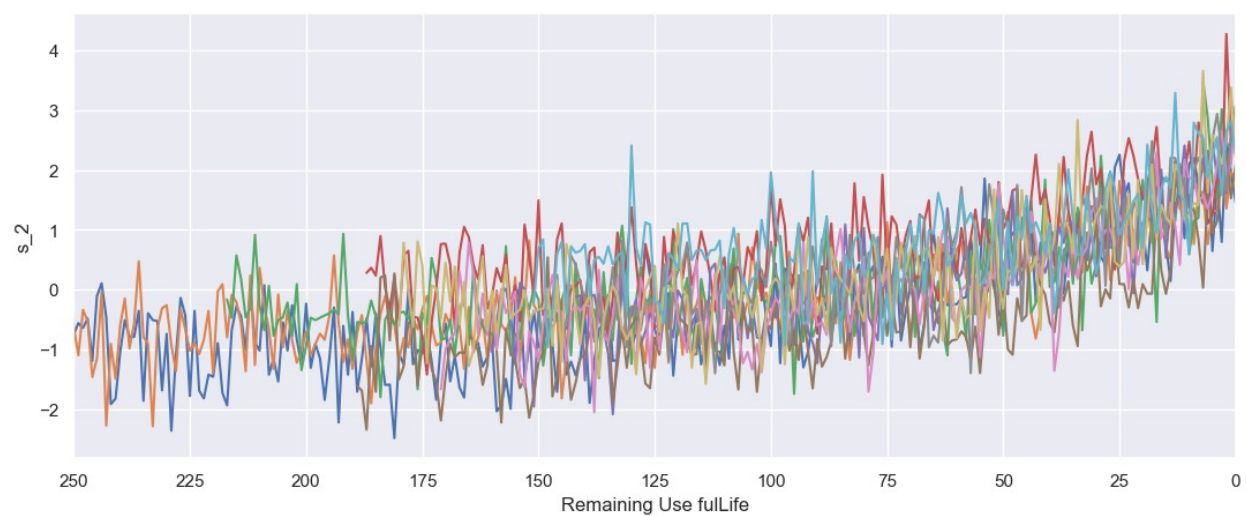
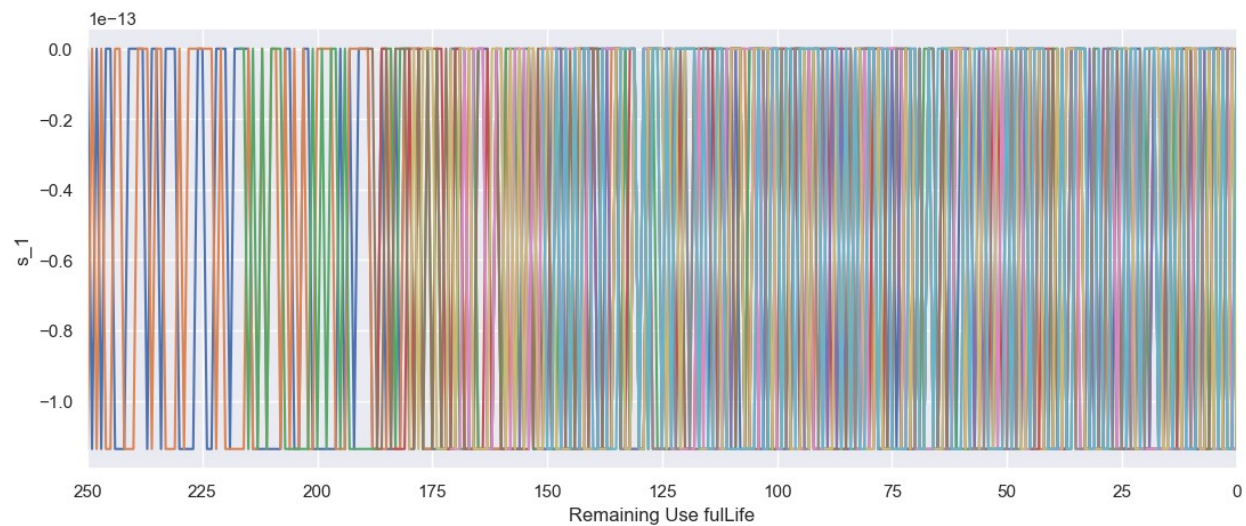
```

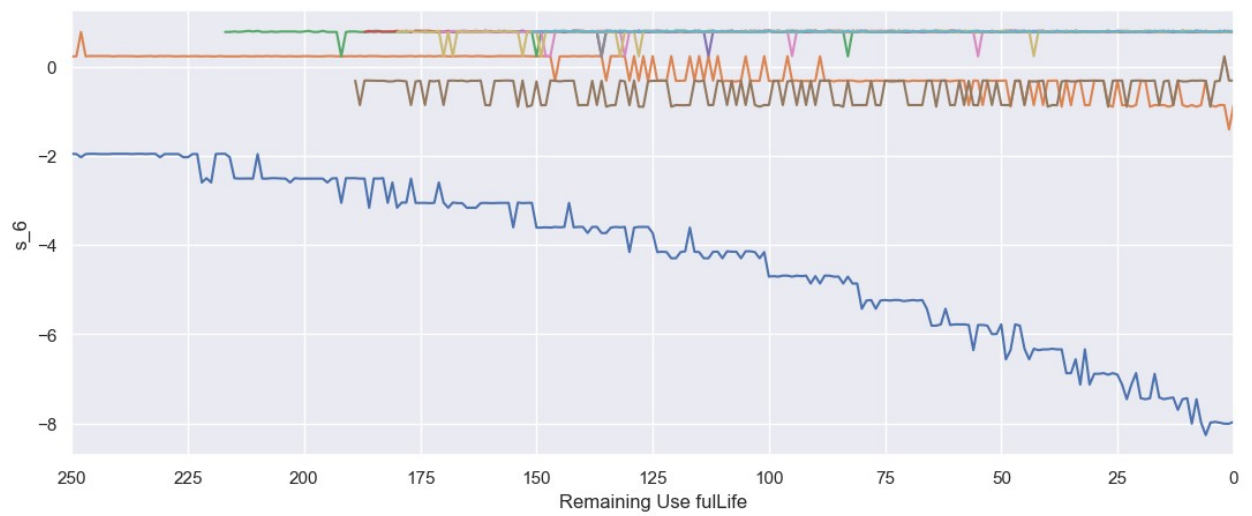
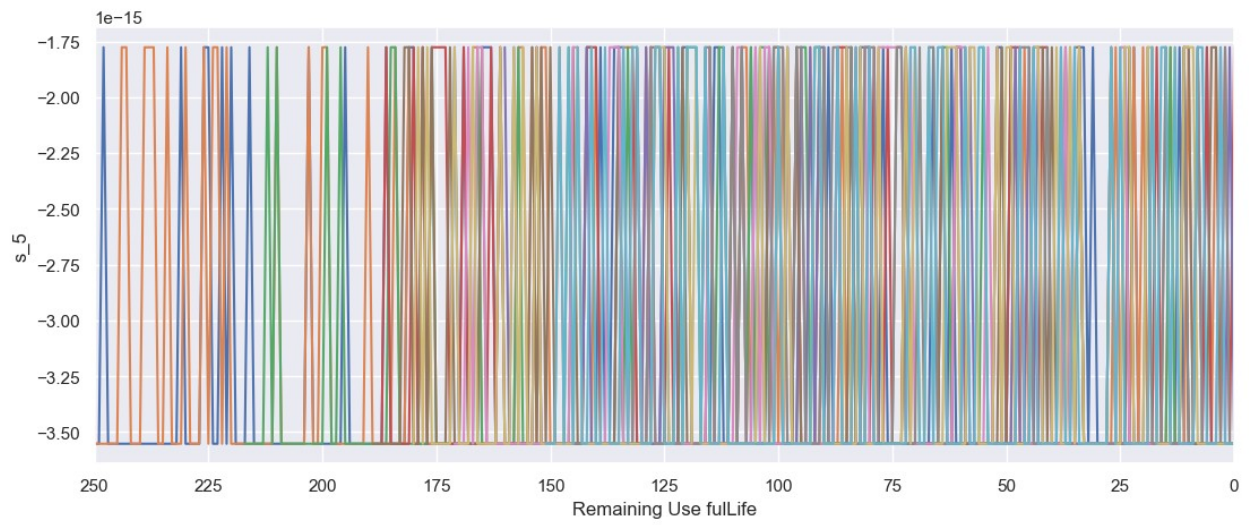
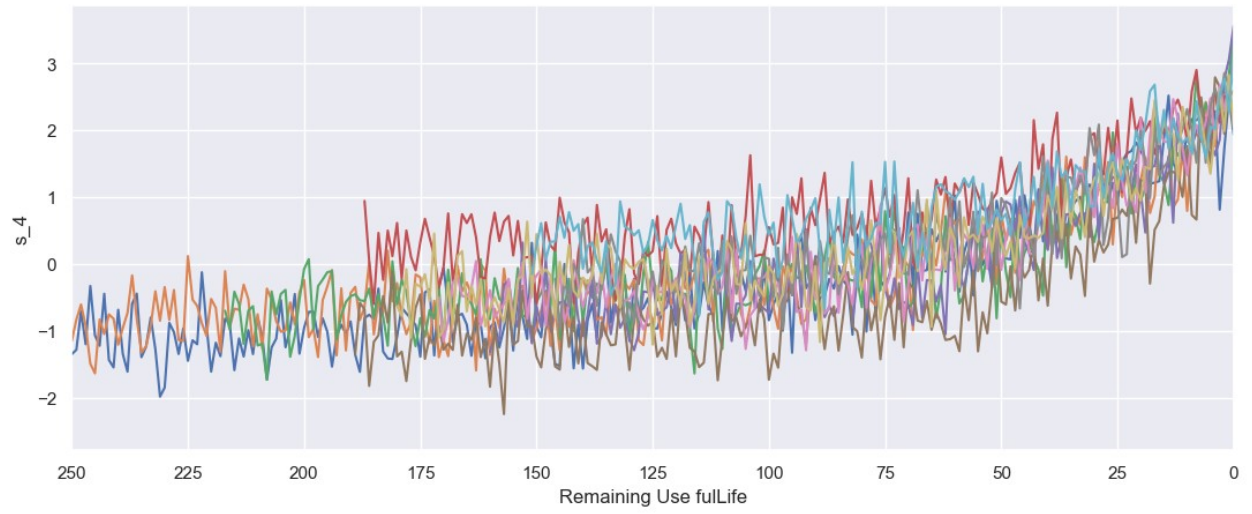
def condition_scaler(df_train, df_test, sensor_names):
    # apply operating condition specific scaling
    scaler = StandardScaler()
    for condition in df_train['op_cond'].unique():
        scaler.fit(df_train.loc[df_train['op_cond']==condition,
sensor_names])
        df_train.loc[df_train['op_cond']==condition, sensor_names] =
scaler.transform(df_train.loc[df_train['op_cond']==condition,
sensor_names])
        df_test.loc[df_test['op_cond']==condition, sensor_names] =
scaler.transform(df_test.loc[df_test['op_cond']==condition,
sensor_names])
    return df_train, df_test
X_train_condition = add_operating_condition(train)
X_test_condition = add_operating_condition(X_test)

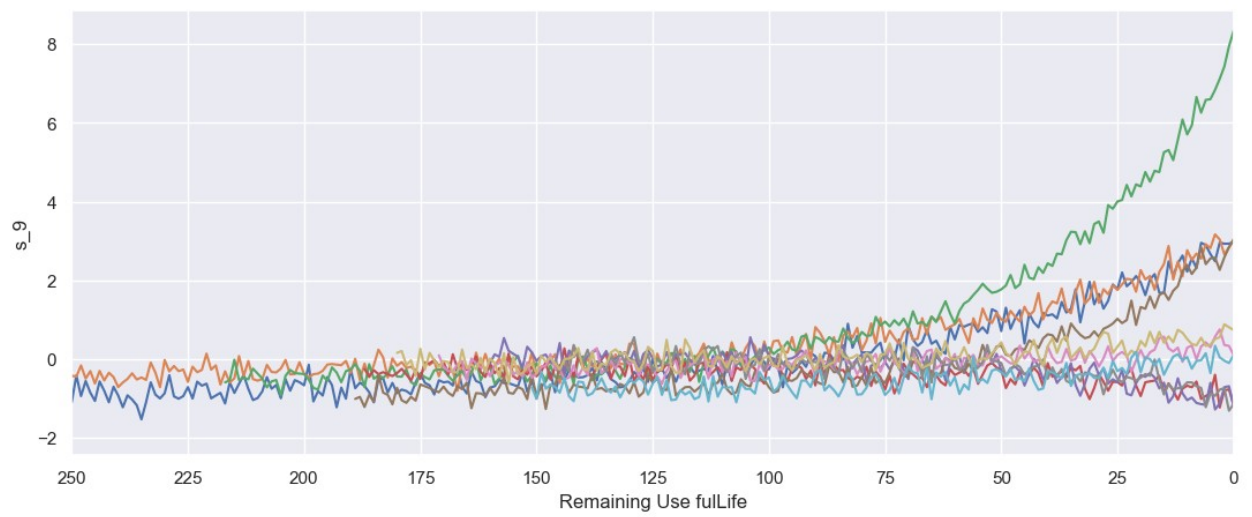
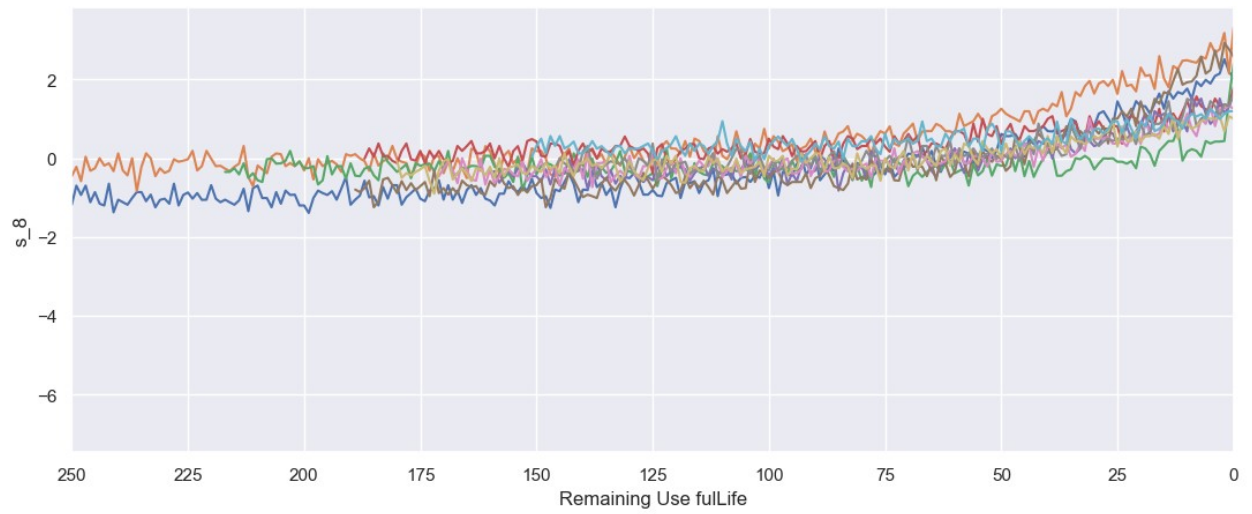
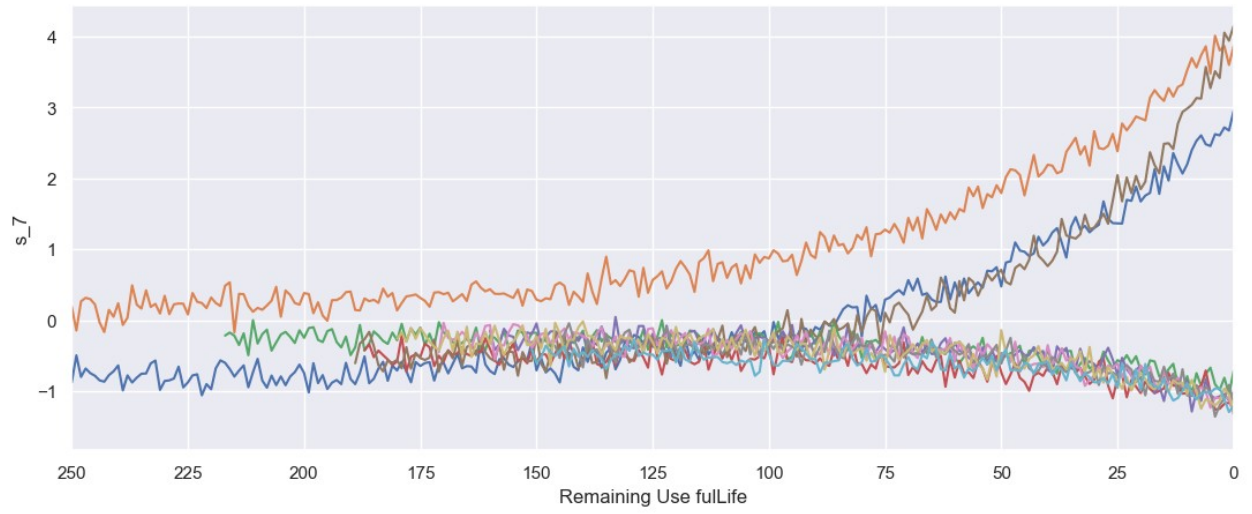
X_train_condition_scaled, X_test_condition_scaled =
condition_scaler(X_train_condition, X_test_condition, sensor_names)
def plot_signal(df, signal_name, unit_nr=None):
    plt.figure(figsize=(13,5))

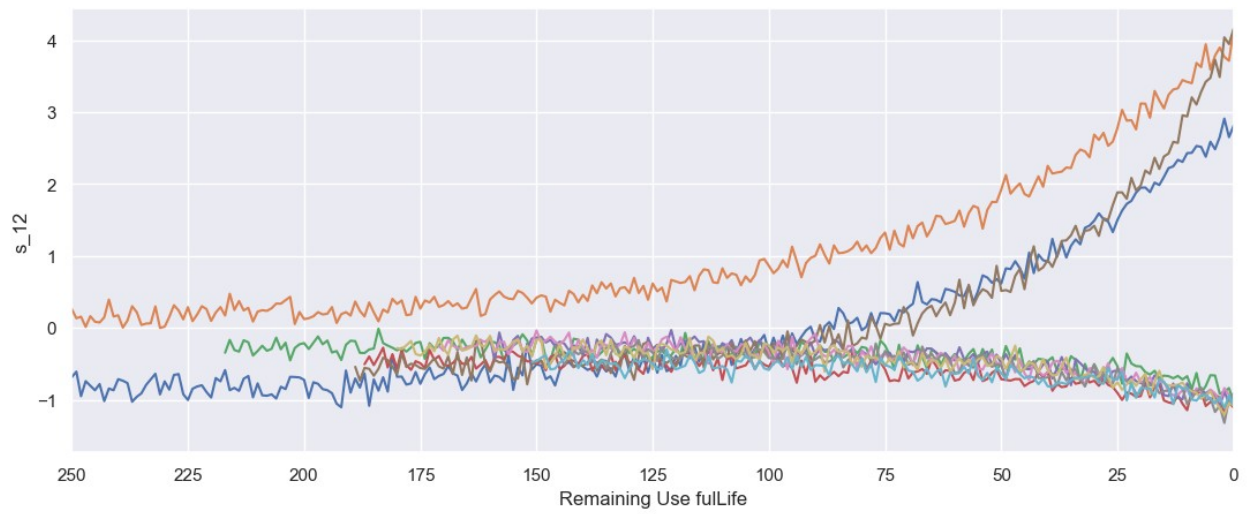
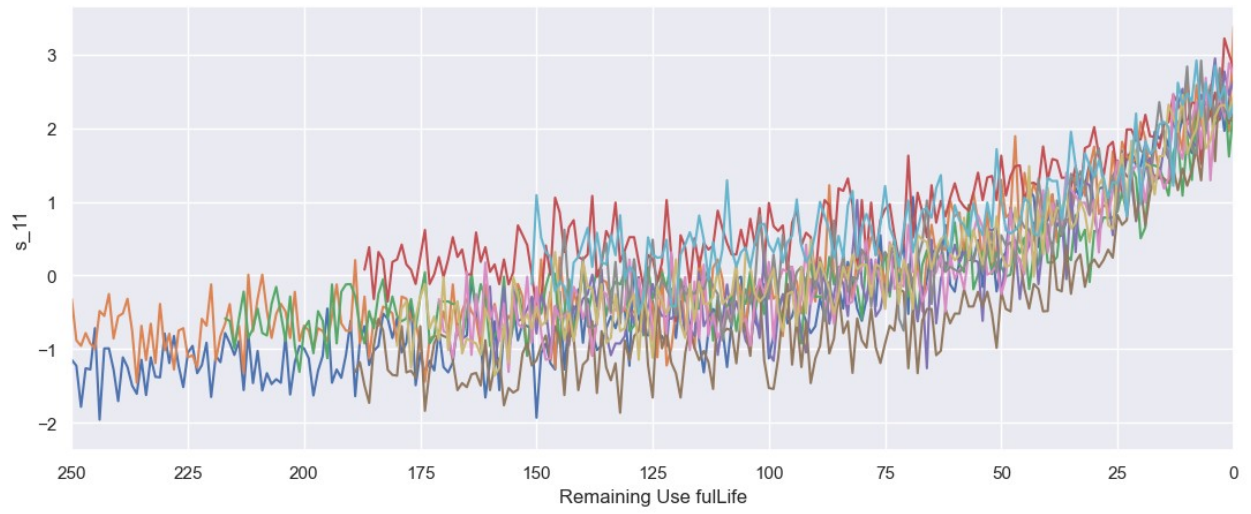
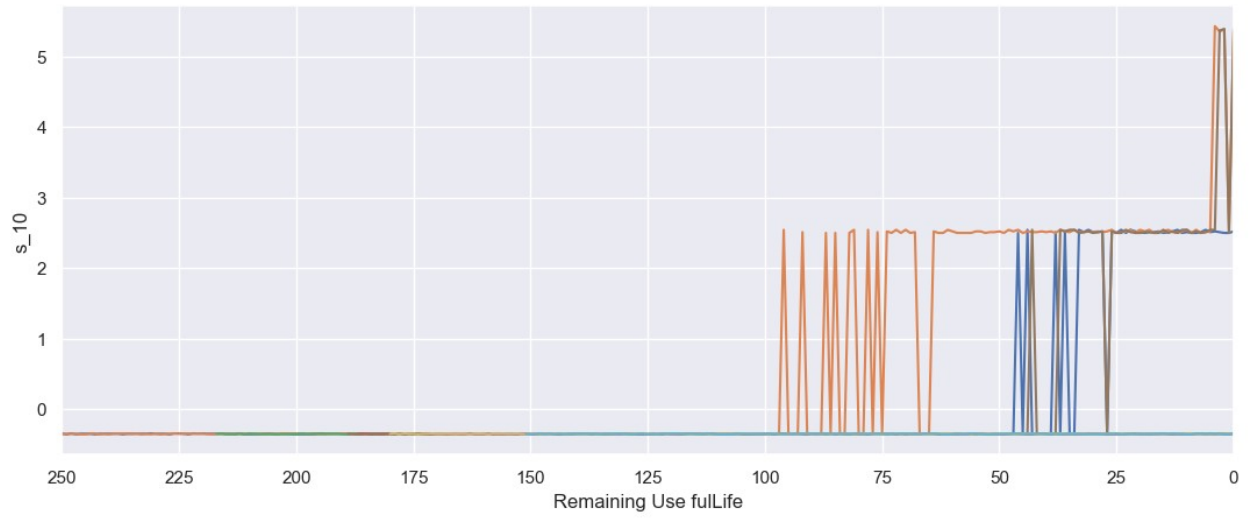
    if unit_nr:
        plt.plot('RUL', signal_name,
                 data=df[df['unit_nr']==unit_nr])
    else:
        for i in train['unit_nr'].unique():
            if (i % 10 == 0): # only plotting every 10th unit_nr
                plt.plot('RUL', signal_name,
                         data=df[df['unit_nr']==i])
        plt.xlim(250, 0) # reverse the x-axis so RUL counts down to zero
        plt.xticks(np.arange(0, 275, 25))
        plt.ylabel(signal_name)
        plt.xlabel('Remaining Use fullLife')
        plt.show()
for sensor in sensor_names:
    plot_signal(X_train_condition_scaled, sensor)

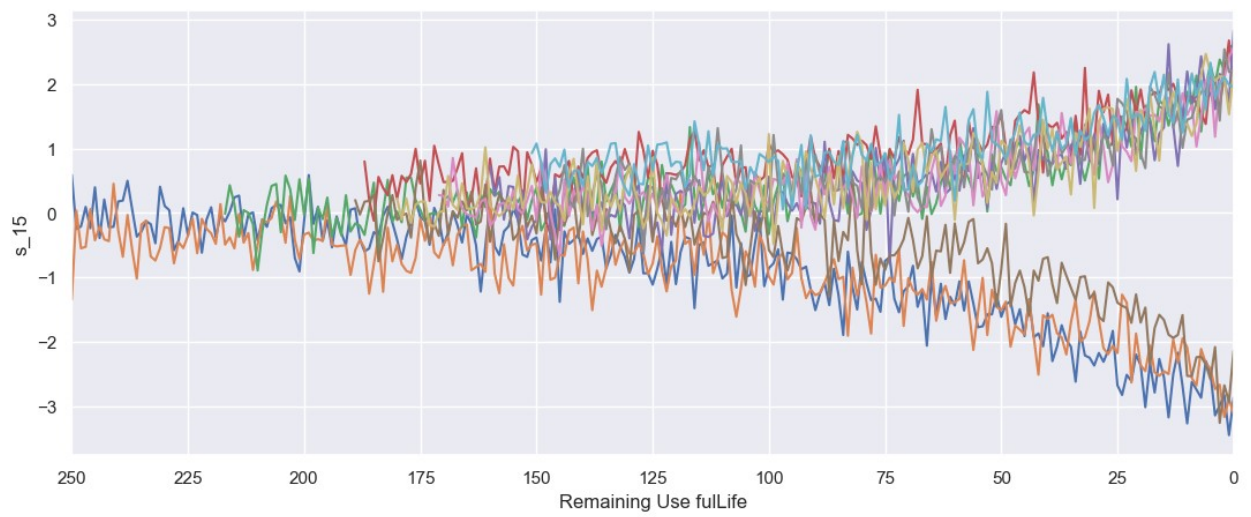
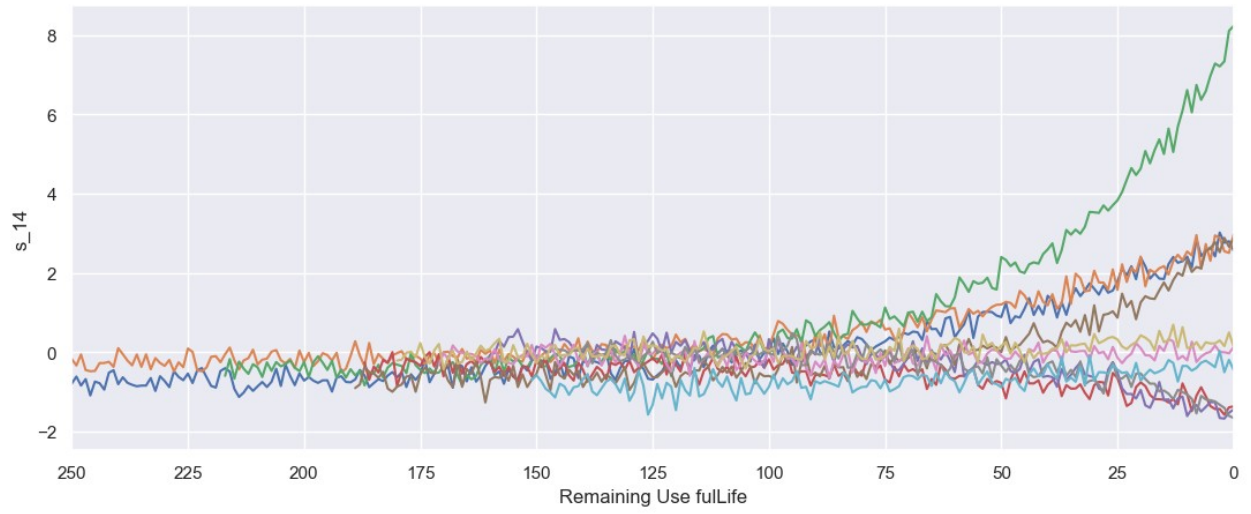
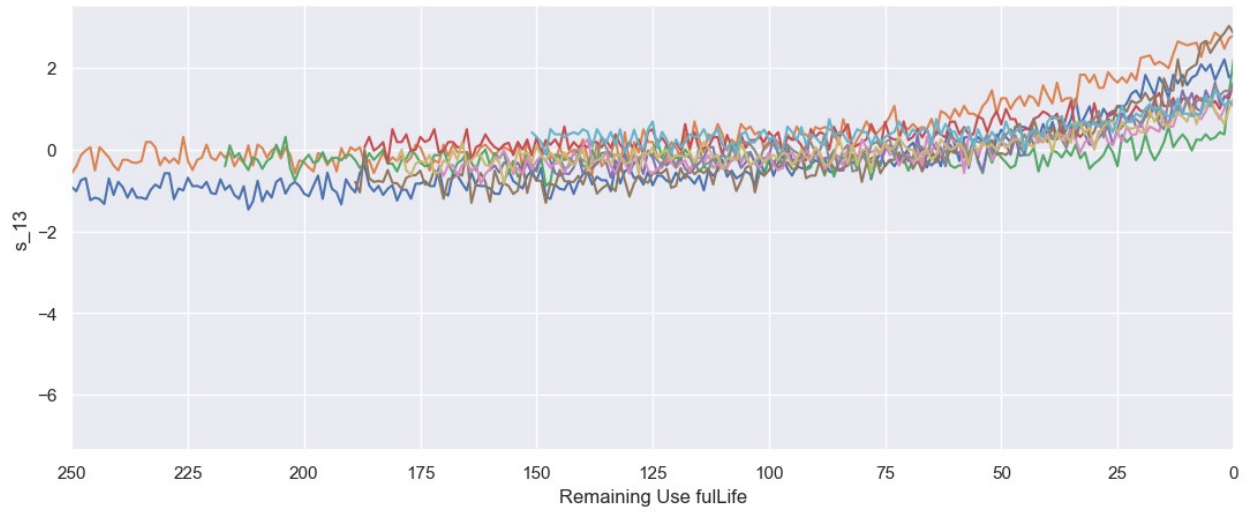
```

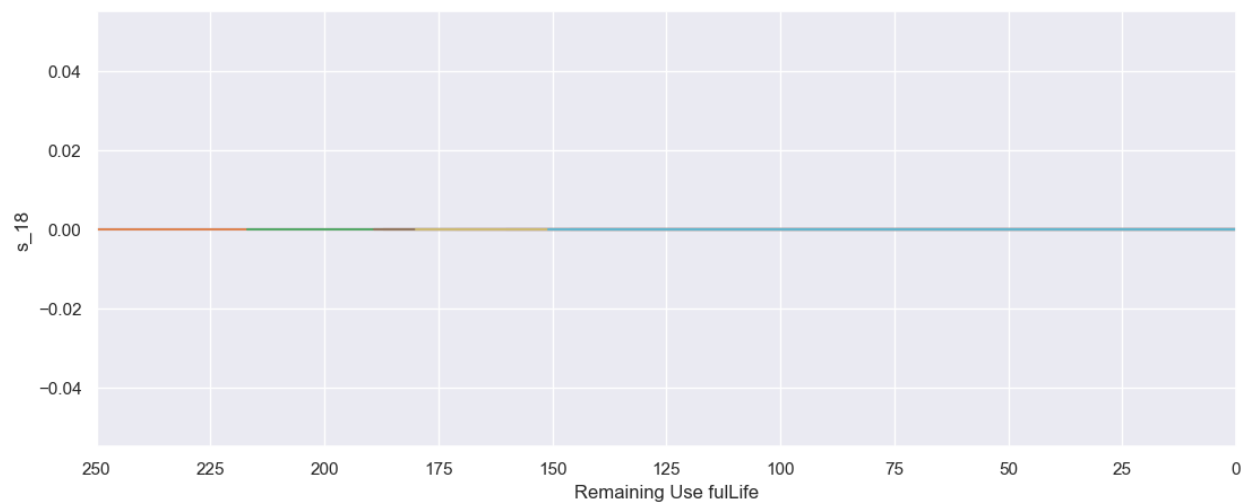
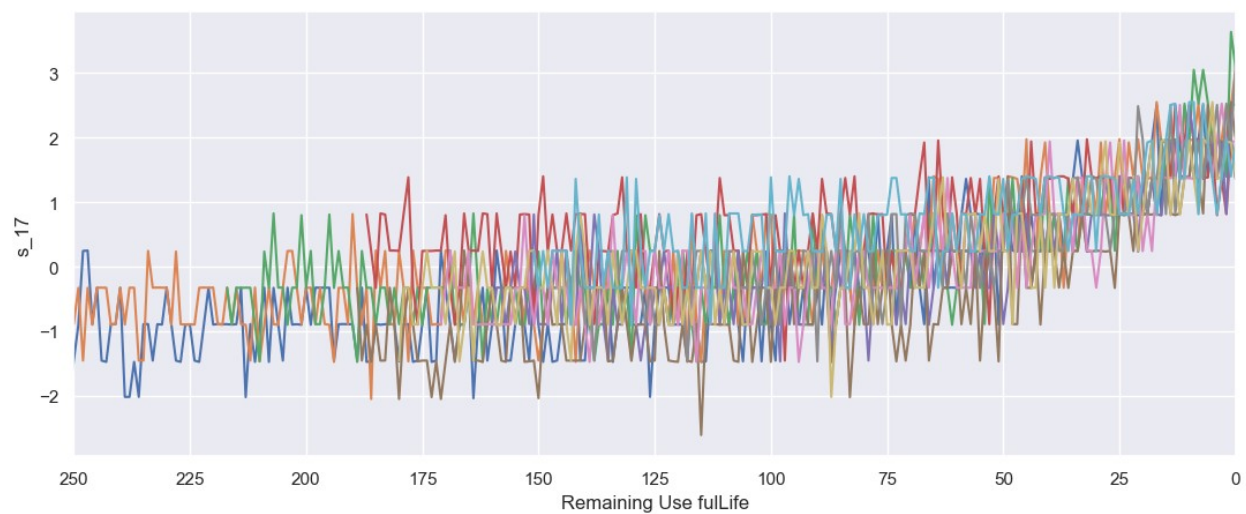
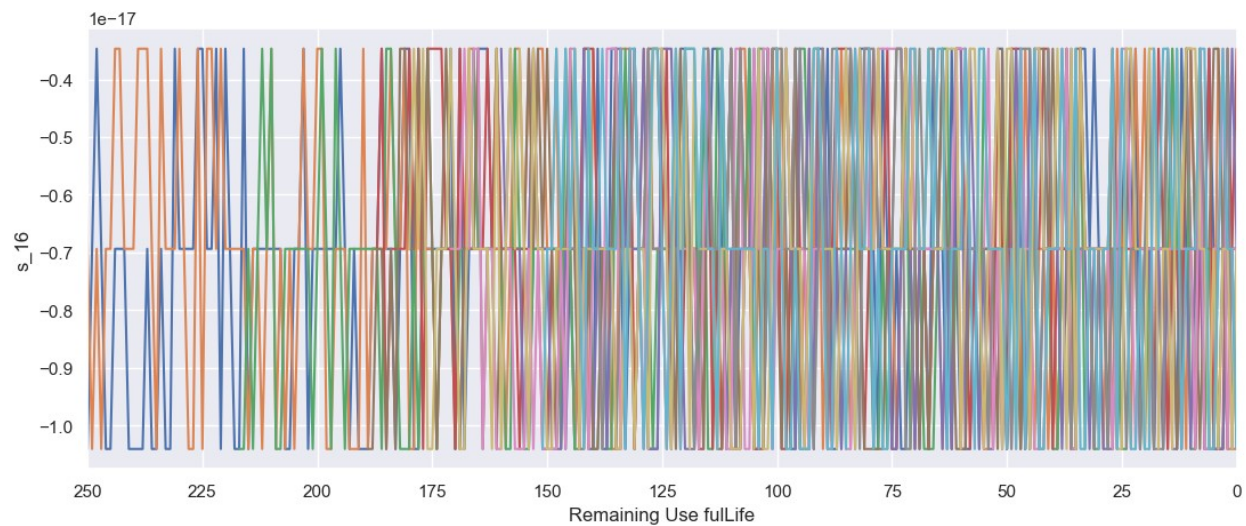


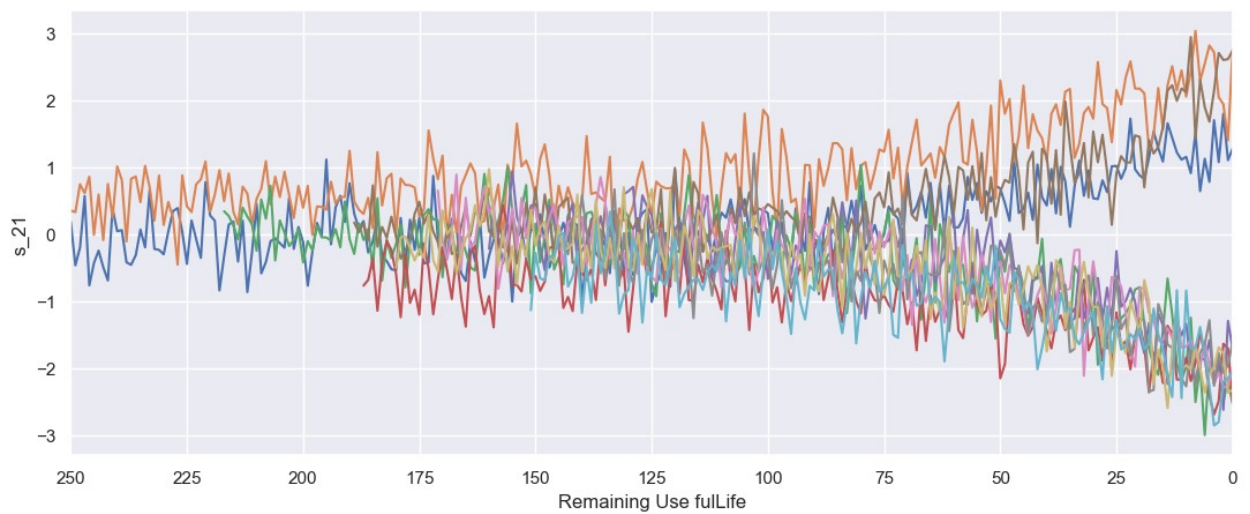
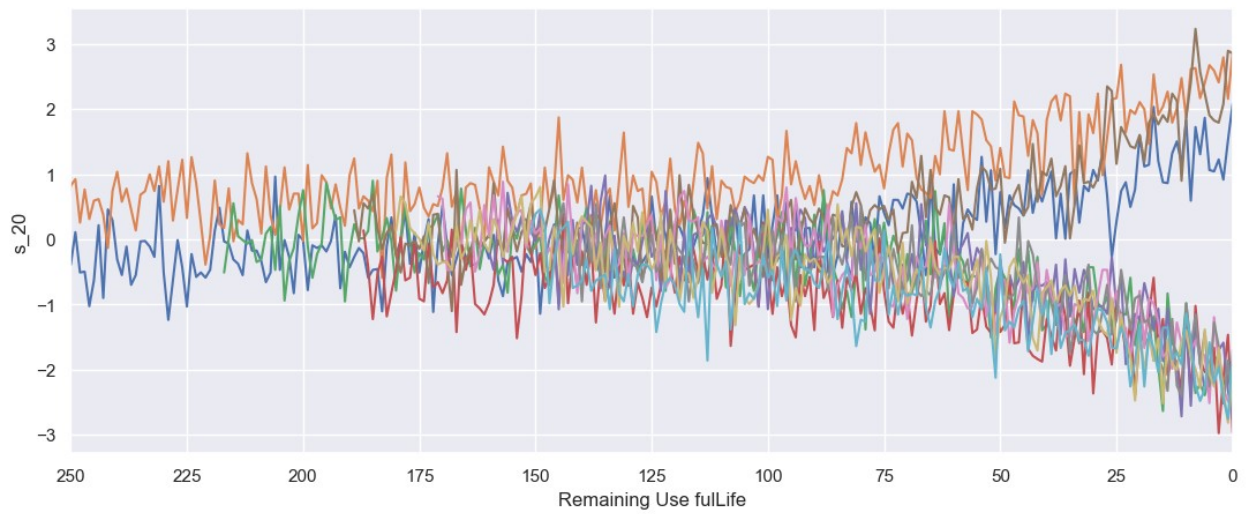
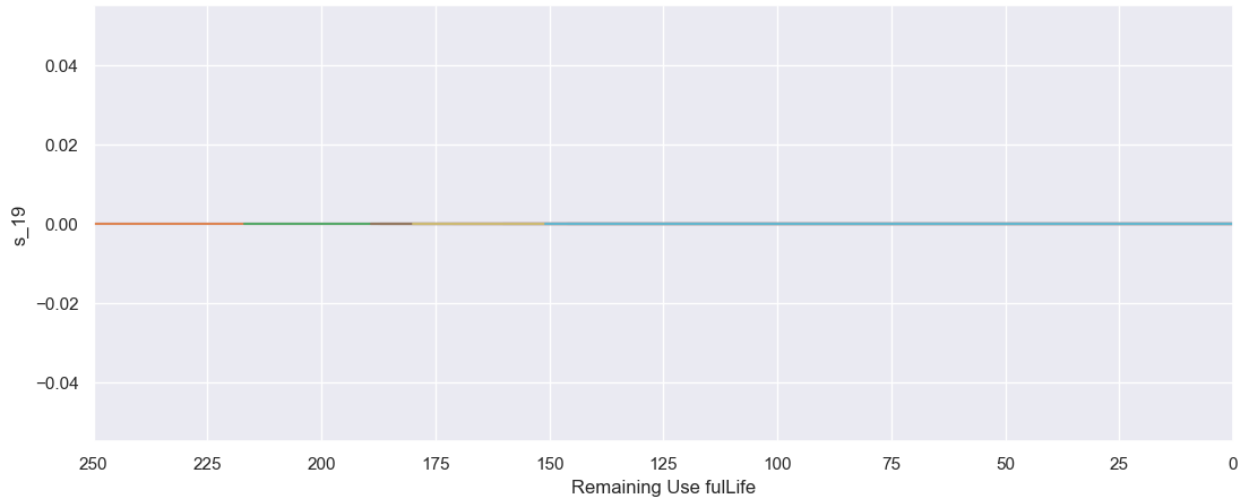












```
remaining_sensors = ['s_2', 's_3', 's_4', 's_7', 's_8', 's_9',  
                    's_11', 's_12', 's_13', 's_14', 's_15', 's_17', 's_20', 's_21']
```

```

drop_sensors = [element for element in sensor_names if element not in
remaining_sensors]

def exponential_smoothing(df, sensors, n_samples, alpha=0.4):
    df = df.copy()
    # first, take the exponential weighted mean
    df[sensors] = df.groupby('unit_nr')[sensors].apply(lambda x:
x.ewm(alpha=alpha).mean())

    # second, drop first n_samples of each unit_nr to reduce filter
delay
    def create_mask(data, samples):
        result = np.ones_like(data)
        result[0:samples] = 0
        return result

    mask = df.groupby('unit_nr')['unit_nr'].transform(create_mask,
samples=n_samples).astype(bool)
    df = df[mask]

    return df
gss = GroupShuffleSplit(n_splits=1, train_size=0.80, random_state=42)
# eventhough we set np and tf seeds, gss requires its own seed

def train_val_group_split(X, y, gss, groups, print_groups=True):
    for idx_train, idx_val in gss.split(X, y, groups=groups):
        if print_groups:
            print('train_split_engines', train.iloc[idx_train]
['unit_nr'].unique())
            print('validate_split_engines', train.iloc[idx_val]
['unit_nr'].unique(), '\n')

        X_train_split = X.iloc[idx_train].copy()
        y_train_split = y.iloc[idx_train].copy()
        X_val_split = X.iloc[idx_val].copy()
        y_val_split = y.iloc[idx_val].copy()
        return X_train_split, y_train_split, X_val_split, y_val_split

d = {'unit_nr': [1]*5 + [2]*5,
      'X': [10., 10.2, 10.4, 10.6, 10.8, 20., 20.2, 20.4, 20.6, 20.8],
      'y': [1.4, 1.4, 1.3, 1.2, 1.1, 2.4, 2.4, 2.3, 2.2, 2.1]}
example_df = pd.DataFrame(data=d)
example_df

```

	unit_nr	X	y
0	1	10.0	1.4
1	1	10.2	1.4
2	1	10.4	1.3
3	1	10.6	1.2
4	1	10.8	1.1

5	2	20.0	2.4
6	2	20.2	2.4
7	2	20.4	2.3
8	2	20.6	2.2
9	2	20.8	2.1

```
def gen_train_data(df, sequence_length, columns):
    data = df[columns].values
    num_elements = data.shape[0]

    # -1 and +1 because of Python indexing
    for start, stop in zip(range(0, num_elements-(sequence_length-1)),
range(sequence_length, num_elements+1)):
        yield data[start:stop, :]
gen = gen_train_data(example_df[example_df['unit_nr']==1],
sequence_length=4, columns=['X'])
list(gen)
```

```
[array([[10. ],
        [10.2],
        [10.4],
        [10.6]]),
 array([[10.2],
        [10.4],
        [10.6],
        [10.8]])]
```

```
def gen_data_wrapper(df, sequence_length, columns,
unit_nrs=np.array([])):
    if unit_nrs.size <= 0:
        unit_nrs = df['unit_nr'].unique()

    data_gen = (list(gen_train_data(df[df['unit_nr']==unit_nr],
sequence_length, columns))
                for unit_nr in unit_nrs)
    data_array = np.concatenate(list(data_gen)).astype(np.float32)
    return data_array
data_array = gen_data_wrapper(example_df, sequence_length=4,
columns=['X'])
data_array
```

```
array([[ [10. ],
        [10.2],
        [10.4],
        [10.6]],

       [ [10.2],
        [10.4],
        [10.6],
        [10.8]]],
```

```

[[20. ],
 [20.2],
 [20.4],
 [20.6]],

[[20.2],
 [20.4],
 [20.6],
 [20.8]]], dtype=float32)

def gen_labels(df, sequence_length, label):
    data_matrix = df[label].values
    num_elements = data_matrix.shape[0]

    # -1 because I want to predict the rul of that last row in the
sequence, not the next row
    return data_matrix[sequence_length-1:num_elements, :]

def gen_label_wrapper(df, sequence_length, label,
unit_nrs=np.array([])):
    if unit_nrs.size <= 0:
        unit_nrs = df['unit_nr'].unique()

        label_gen = [gen_labels(df[df['unit_nr']==unit_nr],
sequence_length, label)
                     for unit_nr in unit_nrs]
        label_array = np.concatenate(label_gen).astype(np.float32)
    return label_array
label = gen_labels(example_df[example_df['unit_nr']==1],
sequence_length=4, label=['y'])
label

label_array = gen_label_wrapper(example_df, sequence_length=4,
label=['y'])
label_array

array([[1.2],
 [1.1],
 [2.2],
 [2.1]], dtype=float32)

def gen_test_data(df, sequence_length, columns, mask_value):
    if df.shape[0] < sequence_length:
        data_matrix = np.full(shape=(sequence_length, len(columns)),
fill_value=mask_value) # pad
        idx = data_matrix.shape[0] - df.shape[0]
        data_matrix[idx:,:] = df[columns].values # fill with
available data
    else:
        data_matrix = df[columns].values

```

```

    # specifically yield the last possible sequence
    stop = num_elements = data_matrix.shape[0]
    start = stop - sequence_length
    for i in list(range(1)):
        yield data_matrix[start:stop, :]
# notice the masked value I passed is a float,
# it should be the same datatype as the data you're working with
gen = (list(gen_test_data(example_df[example_df['unit_nr']==unit_nr],
sequence_length=6, columns=['X'], mask_value=-99.))
        for unit_nr in example_df['unit_nr'].unique())
next(gen)

[array([[ -99. ],
        [ 10. ],
        [ 10.2],
        [ 10.4],
        [ 10.6],
        [ 10.8]])]

import kerastuner as kt
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, LSTM, Conv1D,
MaxPooling1D, Flatten, Dropout, Concatenate, Masking
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import regularizers
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint,
ReduceLROnPlateau
from sklearn.model_selection import GroupShuffleSplit
import copy
import os
from tensorflow.keras.models import load_model

sequence_length = 20
num_features = 14
train['RUL'].clip(upper=125, inplace=True)

X_train_interim = add_operating_condition(train.drop(drop_sensors,
axis=1))
X_test_interim = add_operating_condition(test.drop(drop_sensors,
axis=1))

X_train_interim, X_test_interim = condition_scaler(X_train_interim,
X_test_interim, remaining_sensors)

X_train_interim = exponential_smoothing(X_train_interim,
remaining_sensors, 0, 0.4)
X_test_interim = exponential_smoothing(X_test_interim,
remaining_sensors, 0, 0.4)

```

```

# train-val split
gss = GroupShuffleSplit(n_splits=1, train_size=0.80, random_state=42)
for train_unit, val_unit in
gss.split(X_train_interim['unit_nr'].unique(),
groups=X_train_interim['unit_nr'].unique()):
    train_unit = X_train_interim['unit_nr'].unique()[train_unit] #
gss returns indexes and index starts at 1
    val_unit = X_train_interim['unit_nr'].unique()[val_unit]

    train_split_array = gen_data_wrapper(X_train_interim,
sequence_length, remaining_sensors, train_unit)
    train_split_label = gen_label_wrapper(X_train_interim,
sequence_length, ['RUL'], train_unit)

    val_split_array = gen_data_wrapper(X_train_interim,
sequence_length, remaining_sensors, val_unit)
    val_split_label = gen_label_wrapper(X_train_interim,
sequence_length, ['RUL'], val_unit)

# create sequences train, test
train_array = gen_data_wrapper(X_train_interim, sequence_length,
remaining_sensors)
label_array = gen_label_wrapper(X_train_interim, sequence_length,
['RUL'])

test_gen =
(list(gen_test_data(X_test_interim[X_test_interim['unit_nr']==unit_nr]
, sequence_length, remaining_sensors, -99.))
    for unit_nr in X_test_interim['unit_nr'].unique())
test_array = np.concatenate(list(test_gen)).astype(np.float32)
input_shape=(sequence_length, train_array.shape[2])
# Define input branches
input_cnn = Input(shape=(sequence_length, num_features))
input_lstm = Input(shape=(sequence_length, num_features))

# Define GA Tuner
# GA Tuner extending RandomSearch
class GATuner(keras.tuners.RandomSearch):
    def run_trial(self, trial, *args, **kwargs):
        hp = trial.hyperparameters
        model = self.hypermodel.build(hp)
        # Extract the validation data
        fit_args = copy.deepcopy(args)
        fit_kwargs = copy.deepcopy(kwargs)
        val_data = fit_kwargs.pop('validation_data', None)
        epochs = fit_kwargs.pop('epochs', None)

        # Fit the model
        if val_data:
            history = model.fit(*fit_args, validation_data=val_data,

```

```

epochs=epochs, **fit_kwargs)
    else:
        history = model.fit(*fit_args, epochs=epochs,
**fit_kwargs)

        # Evaluate the model
        val_loss = history.history['val_loss'][-1]
        self.oracle.update_trial(trial.trial_id, {'val_loss':
val_loss})
        self.save_model(trial.trial_id, model)
    def save_model(self, trial_id, model, step=0):
        """Saves the model for a given trial."""
        fname = os.path.join(self.get_trial_dir(trial_id), 'model.h5')
        try:
            model.save(fname)
            print(f"Model saved to {fname}") # Adding logging
        except Exception as e:
            print(f"Error saving model: {e}") # Error handling

# Define the hypermodel
def build_model(hp):
    sequence_length = 20
    num_features = 14

    input_cnn = Input(shape=(sequence_length, num_features))
    input_lstm = Input(shape=(sequence_length, num_features))

    # CNN branch with tunable parameters
    cnn_branch = Masking(mask_value=-99.)(input_cnn)
    for i in range(hp.Int('cnn_layers', 1, 3)):
        cnn_branch = Conv1D(filters=hp.Int(f'cnn_filters_{i}', 32,
128, step=32),
kernel_size=hp.Choice(f'cnn_kernel_size_{i}', [3, 5]),
padding='same',
activation=hp.Choice(f'cnn_activation_{i}', ['relu', 'tanh']))
(cnn_branch)
        if i < hp.Int('cnn_layers', 1, 3) - 1:
            cnn_branch = MaxPooling1D(pool_size=2, padding='same')
(cnn_branch)
        cnn_branch = Flatten()(cnn_branch)
        cnn_branch = Dense(hp.Int('cnn_dense_units', 64, 256, step=64),
activation='relu')(cnn_branch)
        cnn_branch = Dropout(hp.Float('cnn_dropout', 0.2, 0.5))
(cnn_branch)

    # LSTM branch with tunable parameters
    lstm_branch = Masking(mask_value=-99.)(input_lstm)
    for j in range(hp.Int('lstm_layers', 1, 3)):

```

```

        lstm_branch = LSTM(units=hp.Int(f'lstm_units_{j}', 32, 128,
step=32),
                           return_sequences=j < hp.Int('lstm_layers',
1, 3) - 1,
                           activation='tanh')(lstm_branch)
        lstm_branch = Flatten()(lstm_branch)
        lstm_branch = Dense(hp.Int('lstm_dense_units', 64, 256, step=64),
                           activation='relu')(lstm_branch)
        lstm_branch = Dropout(hp.Float('lstm_dropout', 0.2, 0.5))
(lstm_branch)

        # Concatenating the outputs of the two branches
        merged = Concatenate()([cnn_branch, lstm_branch])
        merged = Dense(hp.Int('merged_dense_units', 64, 256, step=64),
activation='relu')(merged)
        merged = Dropout(hp.Float('merged_dropout', 0.2, 0.5))(merged)

        # Output layer
        output = Dense(1, activation='linear')(merged)

        # Create and compile the model
        model = Model(inputs=[input_cnn, input_lstm], outputs=output)
        model.compile(optimizer=Adam(hp.Float('learning_rate', 1e-4, 1e-2,
sampling='log')),
                      loss='mean_squared_error')

        return model

# Instantiate the GA tuner
ga_tuner = GATuner(
    hypermodel=build_model,
    objective='val_loss',
    max_trials=10,
    executions_per_trial=1,
    directory='ga_tuning',
    project_name='ga_tuning_project'
)

# Set up callbacks
es = EarlyStopping(monitor='val_loss', mode='min', verbose=1,
patience=15)
mc = ModelCheckpoint('best_deeper_model.h5', monitor='val_loss',
mode='min', verbose=1, save_best_only=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
patience=5, min_lr=0.00001, verbose=1)

# Add your data preparation code here
# ...

# Run the GA tuner search

```

```
ga_tuner.search([train_split_array, train_split_array],
train_split_label,
                epochs=30,
                validation_data=([val_split_array, val_split_array],
val_split_label),
                batch_size=32,
                callbacks=[es, mc, reduce_lr])
```

```
C:\Users\Kamal\AppData\Local\Temp\ipykernel_34924\3985479474.py:4:
FutureWarning: Not prepending group keys to the result index of
transform-like apply. In the future, the group keys will be included
in the index, regardless of whether the applied function returns a
like-indexed object.
To preserve the previous behavior, use
```

```
>>> .groupby(..., group_keys=False)
```

To adopt the future behavior and silence this warning, use

```
>>> df.groupby(..., group_keys=True)
df[sensors] = df.groupby('unit_nr')[sensors].apply(lambda x:
x.ewm(alpha=alpha).mean())
C:\Users\Kamal\AppData\Local\Temp\ipykernel_34924\3985479474.py:4:
FutureWarning: Not prepending group keys to the result index of
transform-like apply. In the future, the group keys will be included
in the index, regardless of whether the applied function returns a
like-indexed object.
To preserve the previous behavior, use
```

```
>>> .groupby(..., group keys=False)
```

To adopt the future behavior and silence this warning, use

```
>>> .groupby(..., group_keys=True)
df[sensors] = df.groupby('unit_nr')[sensors].apply(lambda x:
x.ewm(alpha=alpha).mean())
```

```
Reloading Tuner from qa tuning\qa tuning project\tuner0.json
```

[illegible]

```
val_split_array], val_split_label),  
    epochs=50,  
    batch_size=32,  
    callbacks=[es, mc, reduce_lr])
```

Epoch 1/50

577/578 [=====>.] - ETA: 0s - loss: 217.5287

Epoch 1: val_loss did not improve from 169.51926

578/578 [=====] - 9s 12ms/step - loss:

217.4848 - val_loss: 169.5193 - lr: 1.0559e-04

Epoch 2/50

574/578 [=====>.] - ETA: 0s - loss: 215.2028

Epoch 2: val_loss did not improve from 169.51926

578/578 [=====] - 6s 10ms/step - loss:

215.5470 - val_loss: 176.6343 - lr: 1.0559e-04

Epoch 3/50

577/578 [=====>.] - ETA: 0s - loss: 213.1402

Epoch 3: val_loss did not improve from 169.51926

578/578 [=====] - 6s 10ms/step - loss:

213.0524 - val_loss: 183.8261 - lr: 1.0559e-04

Epoch 4/50

574/578 [=====>.] - ETA: 0s - loss: 210.6524

Epoch 4: val_loss did not improve from 169.51926

578/578 [=====] - 6s 10ms/step - loss:

211.0866 - val_loss: 184.8186 - lr: 1.0559e-04

Epoch 5/50

577/578 [=====>.] - ETA: 0s - loss: 207.4260

Epoch 5: val_loss did not improve from 169.51926

578/578 [=====] - 6s 10ms/step - loss:

207.4251 - val_loss: 173.1768 - lr: 1.0559e-04

Epoch 6/50

576/578 [=====>.] - ETA: 0s - loss: 204.8187

Epoch 6: val_loss did not improve from 169.51926

Epoch 6: ReduceLROnPlateau reducing learning rate to
5.279277320369147e-05.

578/578 [=====] - 6s 10ms/step - loss:

204.9743 - val_loss: 180.3233 - lr: 1.0559e-04

Epoch 7/50

574/578 [=====>.] - ETA: 0s - loss: 199.8555

Epoch 7: val_loss did not improve from 169.51926

578/578 [=====] - 6s 10ms/step - loss:

200.3526 - val_loss: 176.7569 - lr: 5.2793e-05

Epoch 8/50

578/578 [=====] - ETA: 0s - loss: 201.1373

Epoch 8: val_loss did not improve from 169.51926

578/578 [=====] - 6s 10ms/step - loss:

201.1373 - val_loss: 183.0418 - lr: 5.2793e-05

Epoch 9/50

573/578 [=====>.] - ETA: 0s - loss: 199.7077

```

Epoch 9: val_loss did not improve from 169.51926
578/578 [=====] - 6s 10ms/step - loss:
199.5533 - val_loss: 185.5621 - lr: 5.2793e-05
Epoch 10/50
573/578 [=====>.] - ETA: 0s - loss: 202.3499
Epoch 10: val_loss did not improve from 169.51926
578/578 [=====] - 6s 10ms/step - loss:
202.6099 - val_loss: 179.1433 - lr: 5.2793e-05
Epoch 11/50
574/578 [=====>.] - ETA: 0s - loss: 202.2548
Epoch 11: val_loss did not improve from 169.51926

Epoch 11: ReduceLROnPlateau reducing learning rate to
2.6396386601845734e-05.
578/578 [=====] - 6s 10ms/step - loss:
202.2960 - val_loss: 185.7719 - lr: 5.2793e-05
Epoch 12/50
577/578 [=====>.] - ETA: 0s - loss: 198.7721
Epoch 12: val_loss did not improve from 169.51926
578/578 [=====] - 6s 10ms/step - loss:
198.8139 - val_loss: 178.5389 - lr: 2.6396e-05
Epoch 13/50
574/578 [=====>.] - ETA: 0s - loss: 198.8280
Epoch 13: val_loss did not improve from 169.51926
578/578 [=====] - 6s 10ms/step - loss:
198.7422 - val_loss: 182.7960 - lr: 2.6396e-05
Epoch 14/50
578/578 [=====] - ETA: 0s - loss: 198.9653
Epoch 14: val_loss did not improve from 169.51926
578/578 [=====] - 6s 10ms/step - loss:
198.9653 - val_loss: 179.9021 - lr: 2.6396e-05
Epoch 15/50
576/578 [=====>.] - ETA: 0s - loss: 197.3991
Epoch 15: val_loss did not improve from 169.51926
578/578 [=====] - 6s 10ms/step - loss:
197.3901 - val_loss: 186.4835 - lr: 2.6396e-05
Epoch 16/50
573/578 [=====>.] - ETA: 0s - loss: 196.6480
Epoch 16: val_loss did not improve from 169.51926

Epoch 16: ReduceLROnPlateau reducing learning rate to
1.3198193300922867e-05.
578/578 [=====] - 6s 10ms/step - loss:
196.5165 - val_loss: 182.0254 - lr: 2.6396e-05
Epoch 16: early stopping

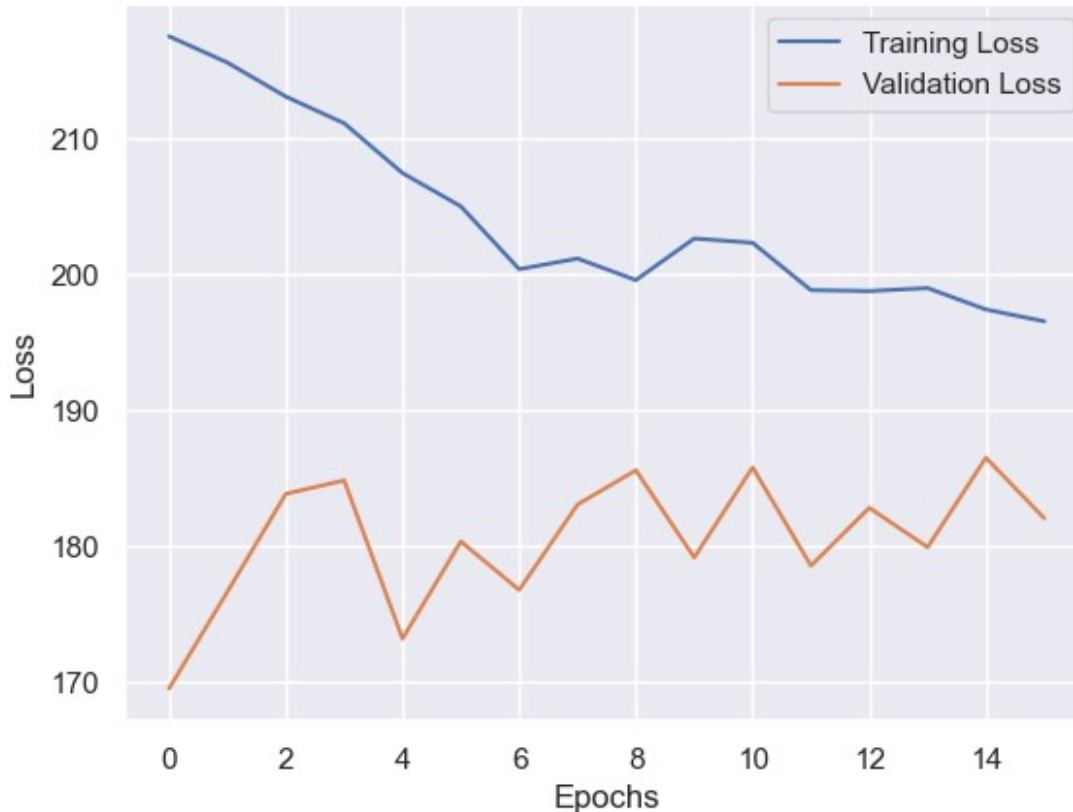
```

```

import numpy as np
import matplotlib.pyplot as plt

```

```
plt.plot(best_history.history['loss'], label='Training Loss')
plt.plot(best_history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



```
y_hat_train = best_model.predict([train_split_array,
train_split_array])
y_hat_val = best_model.predict([val_split_array, val_split_array])
y_hat_test = best_model.predict([test_array, test_array])
evaluate(train_split_label, y_hat_train, 'train')
evaluate(val_split_label, y_hat_val, 'validation')
evaluate(y_test, y_hat_test, 'test')
```

```
578/578 [=====] - 5s 6ms/step
136/136 [=====] - 1s 6ms/step
4/4 [=====] - 0s 4ms/step
train set RMSE:9.042309761047363, R2:0.9516118464634634
validation set RMSE:13.491677284240723, R2:0.8941964086528876
test set RMSE:15.40213225320335, R2:0.8615627858342199
```